PreAct: A Verified Self-Extending Executable Code Corpus for Computer-Using Agents

PreAct Authors[1]
19pine.ai

April 27, 2026

Abstract

Current Computer-Using Agents (CUAs) re-derive already-known action sequences on every invocation, paying full LLM-inference cost even for tasks they have completed before. This "rerun crisis" is both economically wasteful (4–5 seconds of vision-token processing per step) and architecturally unsatisfying: agents accumulate experience but cannot directly execute it.

We present PreAct, a CUA harness that grows a *self-extending executable code corpus*: state-machine programs that are simultaneously the compiled artifact of a successful trajectory *and* the runtime program for subsequent invocations. The harness is a verified compile-extend loop: when CUA succeeds on a task, the live trace is compiled into a state-machine program. When RPA replay later fails or partial-replays, the harness falls through to CUA, generates a fresh state machine, and (passing a *verify-before-store gate*) extends or replaces the corpus. The artifact is the runtime: what the agent "remembers" is what it can directly execute.

We validate two load-bearing claims at multi-seed paper-grade rigor: **(1) Verify-before-store is necessary for monotonicity.** A 2×2 ablation (cold/warm × gate ON/OFF) on AndroidWorld official-15 with n=5 seeds shows gate-ON cold-to-warm $\Delta = +1.2 \pm 0.45$ tasks (all five monotonic) versus gate-OFF $\Delta = -1.4 \pm 0.89$ (all five regress); difference-of-deltas 2.6 tasks (17.3 pp), zero inversions across ten paired observations (sign-test $p < 0.001$). Cross-platform replication on OSWorld test_tiny (n=2) shows identical gate-OFF regression of −3 tasks (−50 pp). Five reproducible cov=100%/score=0 *smoking-gun* replays document the exact lossy-compile pattern the gate filters. **(2) Cold→warm monotonicity holds.** Three independent seeds (rag_db reset per seed) all show monotonic warm refinement, mean $\Delta = +1.33 \pm 0.58$ tasks. Cross-model replication confirms parity is not Claude-specific: Gemini 3 Flash (3-seed mean) and Claude Sonnet 4.6 both achieve 73.3% on AndroidWorld official-15 with an identical stable failure set, implicating harness-level deterministic UI failures rather than model-capability differences.

Equally informative are the *negative* findings: code-level runtime guardrails are statistically aggregate-neutral at n=5 (cold means identical, 10.2 = 10.2); prompt-level Pre-Act content is dead code in the production CUA path. The harness—not prompts or guardrails—is what produces monotonic refinement.

1 Introduction

1.1 The Rerun Crisis

Computer-Using Agents based on the Observation–Reasoning–Action paradigm [11, 4] re-derive their solutions to familiar tasks at every invocation. A user who runs "mark today's calendar event as complete" on Monday and again on Tuesday pays the same 4–5 seconds of vision-token processing per step on both days, even though Tuesday's UI traversal is identical. This is the

rerun crisis [6]: $O(M \times N)$ LLM cost on tasks the agent has already solved, where M is the user's daily task count and N is the per-task LLM call count.

Two prior research directions attack this gap. **Skill systems** (CUA-Skill, Memento, SAGE [9]) save agent behaviors as parameterized skills; the skills still require an LLM-bound loop at execution time, retaining most of the per-step inference cost. **Compilation systems** (CompiledAI [2], Agentic Compilation, ActionEngine [3]) compile workflows into deterministic code; the deterministic-code regime targets narrow business-logic functions or, in ActionEngine's case, generates flat Python scripts from a state machine that is built via untargeted application crawling rather than goal-directed task execution. Concurrent record/replay systems—Muscle-Mem [7], AgentRR [1], Workflow-Use [5]—store linear action sequences with agent fallback if the cache misses; the stored representation has no formal state verification, no conditional branching, and no incremental refinement.

1.2 PreAct's Position

PreAct introduces a different commitment: **the artifact is the runtime**. State-machine programs in PreAct's corpus are simultaneously the compiled artifact of a successful trajectory *and* the directly-executable runtime program. There is no flat-script generation step (cf. ActionEngine), no LLM-bound execution loop (cf. skill systems), no linear-list fragility (cf. Muscle-Mem). When the harness retrieves a state-machine program for a task, it walks the graph: each state's verification predicate is checked against the live UI, and each transition's action is executed. If a verification fails or an action errors, the harness falls through to CUA, generates a fresh state machine from the live trace, and (passing the verify-before-store gate) extends or replaces the corpus.

The corpus thus *self-extends*: the harness's static code defines the compile-extend-replace loop, but the executable state-machine corpus grows monotonically through verified compile cycles. This is closer to a self-modifying executable code corpus than to a passive cache of past actions.

1.3 Contributions

We present and empirically validate the following contributions:

1. **State-machine-as-executable architecture** (§3). The state-machine program produced by compilation *is* the runtime artifact: the harness walks the graph at execution time rather than running generated flat Python.
2. **Self-extending executable code corpus** (§3, §4). The corpus grows through a verified compile-extend-replace loop driven by RPA-replay-failure detection.
3. **Verify-before-store gate** (§3, §4.3). A *double gate* (`replay.success` $\wedge$ `score` ≥ 1.0) filters lossy compiles before they enter the corpus. *Empirically necessary for monotonicity*: cross-platform n=5+2 AB ablation, sign-test $p < 0.001$, five reproducible cov=100%/score=0 smoking-gun replays.
4. **Cross-platform monotonic refinement** (§4.2). On AndroidWorld and OSWorld, cold→warm refinement holds across multiple seeds with the rag_db reset per seed.

The data refute two paper-v1 implications: prompt-level Pre-Act content is dead code in production (§4.4), and code-level runtime guardrails are statistically aggregate-neutral at n=5. **The harness is the contribution; prompts and guardrails are not load-bearing.**

Table 1: PreAct vs. related approaches.

System	Memory representation	Replay fidelity	Fallback path
Muscle-Mem	Linear tool calls	Direct	Agent on cache miss
AgentRR	Multi-level experiences	Indirect	LLM
Workflow-Use	Linear scripts	Direct	Agent
ActionEngine	State machine via crawl	*Flat-Python generated*	None
PreAct	**State machine**	**The SM IS the runtime**	**CUA + recompile under verify-gate**

2 Related Work

We compare PreAct against four classes of prior work: pure CUA baselines, skill-based systems, compilation-based systems, and concurrent record-replay systems. Table 1 summarizes the comparison along five dimensions.

The architectural commitment that distinguishes PreAct is the second column (*the SM is the runtime*) combined with the fourth (*recompile under verify-gate*). ActionEngine builds a state machine but emits flat Python from it; the state machine is consumed at compile time and the runtime is the generated script. PreAct executes the state machine directly: each state's verification predicate is evaluated against the live UI, and each transition is dispatched to the underlying action backend (XPath click, JSON action, etc.). This direct execution enables (a) per-state verification and graceful fallback, (b) in-place patching when a transition fails, and (c) the verify-before-store gate (which we will argue is empirically necessary for monotonicity).

3 System Architecture

3.1 Overview

PreAct's harness comprises four cooperating components: a *Program Selector*, a *State-Machine Replayer*, a *CUA Fallback*, and a *Verify-Before-Store Gate*. The data structure they share is the *Program Corpus*, a content-addressed store of state-machine programs (RPA programs) keyed by a dedup signature.

The high-level loop for any user task T is:

1. **Select.** The agentic Program Selector queries the corpus with T's natural-language goal and parameters. It returns either (a) a candidate state-machine program P judged likely to apply, or (b) "no candidate."
2. **Replay.** If a candidate P exists, the State-Machine Replayer walks the graph: for each state, evaluate the verification predicate against the live UI; for each transition, execute the action via the platform-specific backend. Coverage and progress are tracked.
3. **Fallback.** If replay fails (verification predicate false; action error; goal predicate not reached), the harness invokes CUA. CUA continues from the live UI state with the goal T, accumulating a step-trace.
4. **Verify and store.** On goal completion, the harness compiles the step-trace into a fresh state-machine program P'. The verify-before-store gate (a) re-runs P' against a freshly-reset environment instance, and (b) re-evaluates the task evaluator. The program is added (or replaces an existing program with matching dedup signature) only if both gates pass.

The harness's static Python code defines this loop. The corpus is the variable that grows. Note that step 4's gate is what enables *monotonic* extension: without it, the corpus accumulates lossy

compiles that "run" under replay (cov=100%) but fail the live evaluator (score=0). Section 4.3 demonstrates this empirically.

3.2 State-Machine Programs

A program is a tuple $P = (S, T, M, V)$:

- S: states. Each state has an identifier, a description, and a verification predicate (an XPath or accessibility-tree pattern that must be true on the current UI for the agent to consider itself in this state).
- T: transitions. Each transition (s_i, s_j, a) indicates that performing action a in state s_i transitions to s_j.
- M: metadata. Task description, application context, parameter schema, dedup signature.
- V: verification predicates over data extraction (`inspect_text` actions for QA-style tasks).

The state-machine representation enables three things flat-script representations cannot: (1) per-state verification (the agent knows when an action did not produce the expected effect), (2) conditional branching within the artifact (a state can have multiple outgoing transitions selected by predicate), and (3) in-place extension (a new state and transition can be inserted without regenerating the entire program).

3.3 The Verify-Before-Store Gate

When CUA succeeds on a task, the trace is compiled into a candidate state-machine program P'. The gate ensures P' is monotonically faithful before storage:

1. Reset the environment to the same initial state used for the live run.
2. Replay P' on the freshly-reset environment.
3. Re-evaluate the task using the benchmark's evaluator (`verify_score` ≥ 1.0).
4. Check that the replay itself reported `success` (i.e., reached the terminal state without error).

The double gate (`replay_ok` $\wedge$ `verify_score` ≥ 1.0) is essential. We initially used a single gate (`verify_score` ≥ 1.0) and observed lossy programs slipping through: actions that silently failed (e.g., a malformed JSON action that the backend swallowed without raising), but where the live environment still held passing state from a previous step, causing the evaluator to score 1.0. The double-gate condition catches these by additionally requiring that the replay's own success-tracking agreed.

The opposite failure mode—`replay_ok=True` with `verify_score=0.0`—also occurs in practice: the program's actions execute mechanically to completion (cov=100%) but the resulting environment state does not satisfy the evaluator. We document this as the *cov=100%/score=0 lossy-replay* pattern; §4.3 reports five reproducible cases across two platforms.

3.4 The Agentic Program Selector

The Program Selector is itself an LLM-driven component. We deliberately avoided embedding-based RAG: program descriptions are short, parameter schemas are structured, and the discrimination problem (does this stored program apply to the current task?) is a reasoning task, not a similarity task. The selector receives the goal, the available programs (titles + descriptions + parameter schemas), and chooses via a tool call. Implementation details in Appendix A.

Table 2: Cold-to-warm monotonicity, AndroidWorld official-15, n=3 seeds with rag_db reset per seed. All three seeds show monotonic refinement.

Seed	Cold SR	Warm SR	Δ	Mode-shift on warm
42	10/15 (66.7%)	11/15 (73.3%)	+1	8/13 cua→rpa/hybrid
100	9/15 (60.0%)	11/15 (73.3%)	+2	8/11 cua→rpa/hybrid
1337	9/15 (60.0%)	10/15 (66.7%)	+1	5/10 cua→rpa/hybrid
Mean	**9.33 ± 0.58**	**10.67 ± 0.58**	**+1.33 (+8.9 pp)**	All 3 monotonic

3.5 The CUA Fallback

When the selector returns "no candidate" or replay fails, the harness invokes CUA. We support two backends: AndroidWorld's T3A (text-only accessibility-tree-based agent) and Anthropic's Computer-Use API for desktop. *Critically, the production T3A path uses the verbatim upstream T3A prompt with no Pre-Act-specific additions* (we will return to this in §4.4).

4 Empirical Validation

4.1 Setup

Benchmarks. We evaluate on (a) AndroidWorld [8] "official-15" subset (15 tasks across 8 application domains; the curated subset used by the upstream T3A baseline), and (b) OSWorld [10] "test_tiny" subset (6 tasks across Chrome, LibreOffice Calc, and LibreOffice Writer).

Models. For Android CUA, we use Gemini 3 Flash via the multi-provider port; this is roughly 10× cheaper than Claude Sonnet 4.6 at equivalent SR (we confirm this with cross-model replication in §4.2). For OSWorld CUA, we use Claude Sonnet 4.6 via Anthropic's Computer-Use API. For program compilation and selector reasoning, we use Claude Sonnet 4.6 throughout.

Container. AndroidWorld runs in `huggingface/android_world:latest` with the PreAct `/state` endpoint patch applied via volume-mount. OSWorld uses `happysixd/osworld-docker` via the `DesktopEnv` provider.

Multi-seed protocol. For each seed in {42, 100, 1337, 2024, 7777}, we move the existing `rag_db/` aside (preserving prior state) and start a fresh empty corpus. We then run cold (empty corpus) followed by warm (cold-built corpus) on the same task list.

Cost. The total empirical-validation push (this paper's data) ran in approximately 36 hours of autonomous container time at a total LLM cost of $18–20.

4.2 Cold-to-Warm Monotonicity Holds at n=3 Seeds

Table 2 reports cold-to-warm pairs on AndroidWorld official-15 with n=3 seeds and rag_db reset per seed.

The *mode-shift* column quantifies the extent to which warm runs use retrieval rather than fresh CUA: on seed 42, of 13 successfully completed warm tasks, 8 ran in RPA-replay or hybrid (replay-then-CUA) mode; the corresponding cold runs were 13/13 pure-CUA. Mode shift is direct evidence that the corpus is being used.

Cross-model replication. Repeating the cold experiment with Claude Sonnet 4.6 instead of Gemini 3 Flash gives 11/15 = 73.3% on the same task subset, matching the Gemini 3-seed mean exactly. The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) is

Table 3: Verify-gate ablation, AndroidWorld official-15, n=5 seeds with rag_db reset per seed.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	11	10	−1
100	9	11	+2	11	10	−1
1337	9	10	+1	10	9	−1
2024	11	12	+1	11	10	−1
7777	10	11	+1	12	9	−3
Mean	**9.8**	**11.0**	$+1.2 \pm 0.45$	**11.0**	**9.6**	-1.4 ± 0.89

Table 4: Verify-gate ablation, OSWorld test_tiny, n=2 repetitions, Claude Sonnet 4.6.

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/6	5/6	0	5/6	2/6	−3 (−50 pp)
2	5/6	5/6	0	6/6	3/6	−3 (−50 pp)
Mean	**5.0**	**5.0**	**0**	**5.5**	**2.5**	**−3 (−50 pp)**

identical across both backends and all three Gemini seeds, implicating harness-level deterministic UI failures (status-bar stale read on WiFi; scroll-on-seekbar incompatibility on brightness; image-canvas-not-in-a11y-tree on Draw) rather than model-capability differences.

4.3 Verify-Before-Store is Empirically Necessary for Monotonicity

This is the load-bearing experiment. We run a 2×2 ablation (cold/warm × verify-gate ON/OFF) at n=5 seeds on AndroidWorld and at n=2 repetitions on OSWorld test_tiny.

4.3.1 AndroidWorld (n=5 seeds)

Table 3 shows the result.

The diff-of-deltas is $1.2 - (-1.4) = 2.6$ tasks, or 17.3 percentage points. Across the ten paired observations, all five gate-ON pairs are monotonic ($\Delta > 0$) and all five gate-OFF pairs regress ($\Delta < 0$): *zero inversions*. Under the null hypothesis that the gate has no effect, the probability of observing 5/5 deltas with the predicted sign in each condition is $(1/2)^5 \cdot (1/2)^5 = 1/1024$ (sign-test, $p < 0.001$ one-tailed).

4.3.2 OSWorld test_tiny (n=2 reps, Claude Sonnet 4.6)

Table 4 shows the cross-platform replication.

The proportional regression is larger on OSWorld (50 pp) than on Android (17.3 pp). Both reps show identical $\Delta = -3$ regression. We attribute the larger proportional effect to OSWorld's task evaluators being more state-side-effect-dependent (browser history, calc cell formulae, chart properties) than AndroidWorld's evaluators (which often check end-state UI predicates that mechanical replay can re-create).

Table 5: Five reproducible cov=100%/score=0 lossy-replay events under gate-OFF. Each: program executes its action sequence mechanically to completion, but live evaluator returns score 0. Exactly the lossy-compile pattern the verify gate filters.

Platform	Task	Program ID	Mechanism
Android	ContactsAddContact	80bff413	Replay completes; evaluator misses contact
Android	MarkorCreateFolder	(auto)	Replay completes; folder not at expected path
OSWorld	Chrome history clean	8f0fdfa4	Replay completes; browser state mismatches
OSWorld	LibreOffice Calc formula	43188217	Replay completes; cell formula mismatches
OSWorld	LibreOffice Calc chart	c1f04f87	Replay completes; chart properties mismatch

4.3.3 Mechanism: Five Reproducible cov=100%/score=0 Smoking-Gun Replays

The mechanism is fully observable. Across the gate-OFF runs (both platforms), we identify five distinct programs that are stored unverified, then on warm RPA-replay execute the entire action sequence to completion (`coverage`=100%) and report `replay_ok`=True—yet the live evaluator scores them at 0.0. Table 5 lists them.

These five cases are exactly the lossy-compile pattern the verify-gate exists to filter: *the program "runs" but does not accomplish the goal.* Without the gate, they enter the corpus and produce 14%–50% warm-SR regression. With the gate, they are rejected at compile time; the corpus contains only programs that have independently re-passed an evaluation cycle.

4.4 What is *Not* Load-Bearing

Equally informative are the AB experiments that show negative results. We document two: (a) prompt-level Pre-Act content is dead code in production, and (b) code-level runtime guardrails are statistically aggregate-neutral at n=5. A third (c), the dynamic step-budget cap, saves wall time without sacrificing SR.

4.4.1 Prompt-Level Pre-Act Content: Dead Code

The PreAct codebase contains a Pre-Act-specific prompt block (`prompts.py:148-158`) with three guideline bullets (replace-field semantics, scroll-exhaustion, image-content infeasibility caps). Tracing the production CUA path: `agent.py:464` invokes `T3ACUA.run(goal, env, max_steps=...)` *without* the optional `additional_guidelines` parameter. The Pre-Act-specific bullets are never injected into the prompt.

The 73.3% Android SR is therefore achieved with the verbatim upstream T3A prompt and *zero* Pre-Act prompt-level additions. We treat this as a negative finding: prompt-level Pre-Act content is not contributing to the SOTA-parity claim. The contribution of Pre-Act, on Android, is the harness wrapping T3A—not any prompt modification.

4.4.2 Code-Level Runtime Guardrails: Aggregate-Neutral at n=5

We wired an environment variable `PREACT_GUARDRAILS=on/off` that gates three runtime guardrails: (a) double-tap-before-input_text on populated text fields (replacement semantics), (b) image-task no-op cap (force *infeasible* after 4 consecutive no-op actions on image-keyword tasks), and (c) scroll-direction exhaustion notes appended to step summaries. Table 6 reports the n=5 ablation.

Per-task analysis (omitted for space; see findings summary in supplementary materials) reveals

Table 6: Code-level guardrails ablation, AndroidWorld official-15, n=5. Cold means are *identical* (10.2 = 10.2). Warm $\Delta = +0.4$ is well within the standard deviations on each side.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	10	12	+2
100	11	11	0	11	10	−1
1337	9	11	+2	10	10	0
2024	12	11	−1	10	10	0
7777	9	11	+2	10	11	+1
Mean	**10.2**	**11.0**	$+0.8 \pm 1.30$	**10.2**	**10.6**	$+0.4 \pm 1.14$

Table 7: Validity threats and closure status. Eight of nine in-scope threats closed with multi-seed paper-grade rigor.

#	Threat	Status	Evidence
1	Single-run variance	Closed	n=3 cold→warm + n=5 cold-runs
2	Verify-gate ablation	**Closed cross-platform,** n=5+2	Sign-test $p < 0.001$; OSWorld 50 pp; 5 sm
3	Android cold→warm monotonicity	Closed	n=3 with rag_db reset, all monotonic
4	Prompt-guidance inertness	Closed by codebase state	`agent.py:464` omits `additional_guidel`
5	Code-level guardrails	Closed n=5	Aggregate-neutral (cold means 10.2 = 10.
6	Compile-fidelity taxonomy	Closed	5/5 manual agreement per bucket
7	Step-budget AB	Closed n=5	Within ±2 prediction; wall ratio 0.69
8	Platform-coverage	Out of scope	Requires net-new benchmarks
9	Baseline-parity replication	Closed	Cross-model and cross-seed

task-specific effects that cancel out at the aggregate level. The double-tap guardrail materially helps `AudioRecorderRecordAudioWithFileName`—the task involves a populated filename dialog where vanilla `input_text` appends rather than replaces—but is counterbalanced by minor timing penalties on Camera tasks. The Pre-Act SR claim does *not* depend on these runtime guardrails. We retain them in the production code as edge-case protections, but we do not claim them as contributions in this paper.

4.4.3 Step-Budget Cap: Wall-Time-Only

The dynamic step-budget cap `min(max(20, 8c), 30)` (where c is task complexity) was hypothesized to bound wall time without sacrificing SR. We test against an unbounded 60-step fixed budget at n=5 seeds. The cap-A (current default) mean is 9.8 ± 0.84; cap-B (60-step) mean is 11.6 ± 0.55 (cold runs only). The diff is $+1.8 \pm 0.84$, within the validation-plan ±2 prediction interval. The wall-time ratio (cap-A : cap-B) is 0.69, within the plan's 0.7 prediction. The +1.8 SR is paid for by +30% wall time on the FAIL tail—primarily SystemBrightnessMax, which consumes the full 60-step budget under cap-B and still fails (the failure is harness-deterministic; more budget does not help).

4.5 Summary: 8 of 9 Validity Threats Closed

Table 7 maps each of the nine validity threats from §2 to its closure status.

5 Discussion

5.1 Why the Harness is Load-Bearing

The mechanism is: the verify-gate filters lossy compiles before they enter the corpus. "Lossy" here means: action sequences that are mechanically faithful (cov=100% on replay) but do not produce the live evaluator's required end state. Without the gate, every CUA-success-then-compile cycle adds a new program to the corpus regardless of whether that program actually achieves the goal under deterministic re-execution. With the gate, only programs that have independently re-passed enter the corpus.

This matters because the corpus is the input to subsequent retrieval. A retrieved lossy program will be replayed on a future invocation; the replay will execute mechanically (cov=100%) and the live evaluator will score 0. Worse, the harness's selector will continue to retrieve the lossy program until something explicitly removes it. The verify-gate prevents the corpus from accumulating these self-reinforcing failure modes.

5.2 Harness-Deterministic vs. LLM-Driven Failures

Three AndroidWorld tasks (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) and one OSWorld task (LibreOffice Writer macro) fail across all five seeds, both LLM backends (Claude and Gemini), all four guardrail/budget configurations, and both verify-gate conditions. These failures are *harness-deterministic*: they are properties of the UI patterns themselves (status-bar stale reads, scroll-on-seekbar incompatibility, image-canvas-not-in-a11y-tree) rather than properties of the agent's reasoning. They bound the benchmark, not the method.

5.3 Threats to Validity

The closure table (Table 7) lists eight closed threats and one out-of-scope. Beyond those, two threats remain that the paper acknowledges:

Architectural untested. We did not run an ActionEngine-style flat-script baseline or an untargeted-exploration baseline. The architectural commitments (state-machine-as-executable, task-directed recording) are supported by the working implementation across two platforms but not by direct AB comparison.

Generalizability. OSWorld test_tiny has only 6 tasks, and AndroidWorld official-15 has 15. While the verify-gate effect replicates cross-platform, the benchmarks themselves are a small sample of computer-using tasks. We acknowledge platform-coverage bias (#8 in Table 7) as out-of-scope for this paper.

6 Conclusion

PreAct demonstrates that a *verified self-extending executable code corpus* produces monotonic refinement on multi-platform CUA benchmarks. The harness—RAG retrieval, verify-before-store gate, agentic program selector, hybrid replay, and CUA fallback—is the load-bearing contribution: cross-platform n=5+2 ablation shows the gate is empirically necessary for monotonicity, with five reproducible smoking-gun replays documenting the lossy-compile pattern it filters. Equally informative are the *negative* findings: code-level runtime guardrails are aggregate-neutral, and prompt-level Pre-Act content is dead code in production. The harness is what works; the runtime add-ons are not load-bearing.

The architectural commitment that makes this possible is *the artifact is the runtime*: state-machine programs in PreAct's corpus are not flat scripts generated from a state machine but the directly-executable graphs themselves. What the agent "remembers" is what it can directly execute, and the corpus self-extends through verified compile cycles.

Future work: scale to longer-horizon tasks, more diverse benchmarks, and integration with explicit goal-decomposition planners.

References

[1] Anonymous. AgentRR: Multi-level experience replay for agents. *arXiv preprint arXiv:2505.17716*, 2025.

[2] Anonymous. Compiled AI: Compiling agent workflows into deterministic code. Working manuscript, 2025.

[3] Anonymous. ActionEngine: State machine discovery via app crawling. *arXiv preprint arXiv:2602.20502*, 2026. Microsoft Research.

[4] Anthropic. Introducing computer use. https://www.anthropic.com/news/3-5-models-and-computer-use, 2024.

[5] Browser-Use. Workflow-use: Linear script compilation with agent fallback. https://github.com/browser-use/workflow-use, 2025.

[6] Saideep Chundru. The rerun crisis in computer using agents, 2026. Working manuscript.

[7] Pig.dev. Muscle-mem: Tool-call replay with agent fallback. https://github.com/pig-dot-dev/muscle-mem, 2025.

[8] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *ICLR*, 2025.

[9] Yongchao Wang et al. SAGE: Skill-augmented agent architecture. Working manuscript, 2024.

[10] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *NeurIPS Datasets and Benchmarks Track*, 2024.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.

Appendix A. Computer-Action Schema

The full JSON action schema and per-platform backend mappings are described in the project's `DESIGN.md` (§Appendix A), reproduced here for completeness. (Omitted in this draft.)

Appendix B. Per-Task Multi-Seed Data

The complete per-task results across all 32 Android multi-seed runs and 8 OSWorld runs are available in the project's `paper/findings_summary.md`.

Appendix C. Container Patch

The `android_server_patched.py` adds a `/state` endpoint to `huggingface/android_world:latest` that returns base64-encoded screenshots plus serialized accessibility-tree elements, sidestepping the upstream populated-AVD a11y-gRPC initialization wedge.